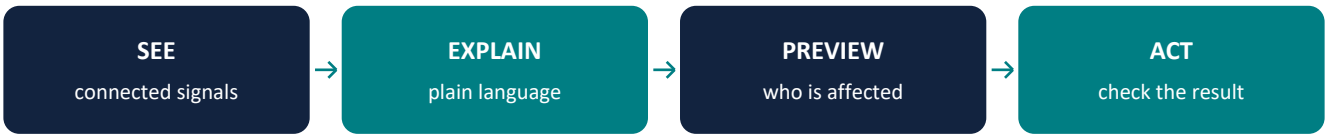
01 / START HERE

Security decisions people can understand

VAJRA Saathi is a security teammate for organizations without a dedicated security team.

An owner should not need to understand security event codes to protect their business. They need to know what happened, what is uncertain, what they can do, and whose work their decision will affect.

Understand the problem. Preview the consequences. Take a verified protective action.



Saathi brings together a small helper on office computers, a phone-friendly web application, a policy engine and an evidence system. It can explain in English, Kannada or Hindi. It records what it did and can attach an independent blockchain timestamp to the evidence.

One product, many organizations

A clinic protects patient-related records. A shop protects billing and customer information. A school protects student records. An NGO protects beneficiary information. The product changes its vocabulary, while the same access-control and verification rules remain underneath.

What this document provides

The first part explains Saathi without assuming technical knowledge. The second connects the screens to the backend, database, computer agent and blockchain proof. The final part explains how to build, test and demonstrate it honestly.

Design edition: 31 October 2026. This is a proposed complete product specification, not a report of completed implementation or a guarantee of perfect security.

02 / READING MAP

A guide for every reader

Start with the human decision. Continue into the machinery that makes it trustworthy.

Pages	What you will understand
3-5	The problem, everyday terminology and one incident from beginning to end.
6-8	Consequence preview, product navigation, organization setup and coverage.
9-12	Threat handling, system architecture, database contracts and policy correctness.
13-16	Windows actions, recovery, AI boundaries, accessibility and preventive health.
17-19	Signed evidence, blockchain timestamps and independent verification.
20-22	Build sequence, evaluation, demonstration and honest deployment limits.
23	Source references and the companion implementation documents.

Three documents, one specification

build.md is the engineering execution plan: what to build first, how services connect, the API inventory, database model and acceptance gates. flow.md is the product walkthrough: screens, user roles, transitions and failure paths. This PDF explains the reasoning and mechanics behind both.

Three claims that must remain separate

- Detection: Saathi observed signs that deserve attention. It may still be uncertain about intent.
- Protection: a particular restriction was applied and its effect was checked within a defined scope.
- Evidence: a retained record matches signed data and, where verified, an externally timestamped commitment.

A valid timestamp cannot prove that a threat was stopped. A successful command cannot prove that every threat is contained.

03 / THE HUMAN PROBLEM

Why an ordinary office needs this

The hardest moment is often deciding what to do without stopping legitimate work.

Small organizations depend on computers and accounts, but the person responsible may be an owner, receptionist, manager or administrator. They may have neither the time nor the expertise to interpret a long security console.

Situation	The decision the person actually faces
Unusual record downloads	Is this a legitimate export? Can I pause only that session?
Repeated wrong passwords	Is someone trying to get in, or did a colleague forget their password?
Unexpected USB storage	What can be restricted now, and what remains accessible?
Former staff login	Which connected systems still recognize this person?
Suspicious message	What looks wrong, what is uncertain, and what should I do next?

What Saathi contributes

It combines relevant observations into a smaller set of decisions. It puts facts and uncertainty in different places. It explains the likely consequences of each supported response, recommends a narrow option where justified, and checks what actually happened after approval.

The first users

An owner sets policy and responsibilities. An approver handles incidents. Staff report suspicious messages. An auditor can inspect permitted history. One person may hold several roles, but the system still checks what each role is allowed to do.

A school is one useful story. The product is designed around the shared needs of small organizations, not around education-specific software.

04 / NO SECURITY BACKGROUND REQUIRED

The idea in everyday language

Think of a watchman, a translator, a careful operator and a record keeper working together.

Saathi helper	What that means
Watchman	Notices selected signals from connected computers and applications.
Translator	Turns technical observations into clear language without pretending uncertainty is certainty.
Careful operator	Shows the effect of a response, checks authority, and performs only supported actions.
Record keeper	Keeps a signed history and a reference that can be checked later.

Eight words you will see

Word	Plain meaning
Account	A person or shared identity allowed to use a system.
Session	One login. The same person can have two sessions on different devices.
Event	An observation recorded by a connected source.
Incident	Related observations grouped into a situation needing attention.
Preview	What a proposed response would stop, limit or leave working.
Probe	A check of the actual state after an action.
Checkpoint	A signed summary identifying a particular version of the evidence history.
Blockchain anchor	An external timestamp commitment used to check that version later.

The helper only observes the defined telemetry and user-submitted messages. It is not intended to record keystrokes, continuously capture screens, or monitor students or patients.

05 / AN ORGANIZATION-NEUTRAL STORY

One incident, beginning to end

Two sessions belong to one staff member. One behaves suspiciously; the other is doing legitimate work.

1. The unusual activity begins

A connected records application receives many download requests from an unfamiliar session. The normal office session is still being used for ordinary work. This could be misuse, or an authorized task that needs checking.

2. Saathi reduces exposure and explains

If the owner previously agreed to temporary throttling, the gateway limits the suspicious session under that policy. Saathi groups the observations and sends a generic notification. The incident explains the rate, unfamiliar-session signal, uncertainty and current limit.

3. The owner compares responses

The phone shows that pausing the suspicious session blocks that session while the office session keeps its existing permissions. Disabling the whole connected account has a broader effect. The preview also says already downloaded files cannot be recalled.

4. Approval is checked against current reality

The owner chooses the narrow response. Saathi rechecks permissions and whether the relevant situation changed since the preview. If it changed, nothing is applied until a fresh preview is approved.

5. The outcome is demonstrated

The suspicious session is denied further protected requests, and the intended legitimate session still works. The UI displays the observed result. If another finding remains unresolved, it says so.

6. Recovery and evidence follow

The owner verifies the staff member, secures access as needed and issues a fresh session. The old token remains rejected. A signed receipt captures the sequence; an independent timestamp can complete later.

The memorable outcome: suspicious access stopped within the connected application, and legitimate work continued.

06 / THE CORE DIFFERENTIATOR

Preview before pressing the button

Every response should explain its consequences before it changes access.

Response	Effect in the example	Important boundary
Pause one session	Stops that session; other permitted sessions stay active.	Only the connected application is covered.
Disable account	Restricts the account according to the connector and its session controls.	Existing sessions need explicit assessment or revocation.
Keep investigating	Makes no additional restriction.	Existing exposure can continue; a temporary limit may expire.

Every preview has six parts

- Stops: operations that become denied.
- Limits: operations that remain possible at a reduced rate or scope.
- Keeps working: known operations whose permission is unchanged.
- Unknown: dependencies or applications Saathi cannot assess.
- Cannot control: already copied data and other hard boundaries.
- Recovery: how access can be restored safely and who must approve it.

Why this is more than reassuring copy

For the reference records application, preview and enforcement use the same policy function. Tests compare the predictions with actual permission transitions. For a Windows action, the preview uses known device posture; a post-action probe checks the real device.

“Nobody else is affected” is justified only within the known connected scope. Missing dependencies must remain visible.

07 / PRODUCT NAVIGATION

How people move through Saathi

A small number of destinations lead to the same consistent action workflow.

Destination	What the person does next
Overview	Open a concern, inspect coverage, or review what was handled.
Incidents	Read facts and uncertainty; compare actions; approve; inspect results.
Devices	See last contact, posture, supported controls and device history.
People and access	Inspect connected sessions, review grants and offboard staff.
Ask Saathi	Submit a message, understand findings and report a concern.
Health	See transparent posture checks and preview supported fixes.
Evidence and reports	Read the record, download a bundle and inspect timestamp status.
Integrations and settings	Manage coverage, roles, language, working hours and policy.



Staff normally land on Ask Saathi. Owners and approvers land on Overview. Auditors land on evidence or permitted reports. A link always checks current membership and role, even if the user already has the page open.

The business records workspace is a separate reference application. Its sessions must not be confused with the owner's security-console login. Full route-by-route navigation is defined in flow.md.

The interface never shows a security action as successful merely because the user clicked a button.

08 / ONBOARDING AND COVERAGE

Setup defines what Saathi can know

Coverage is part of the product, not a footnote at the end of a report.

First-time setup

- Create the organization: type, language, timezone and working hours.
- Invite people: owner, approver, staff and auditor; keep a recovery identity.
- Declare dependencies: shared accounts, critical workstations and business applications.
- Enrol authorized Windows devices using a one-use code.
- Connect applications and distinguish observation from enforcement.
- Choose explicit temporary automation and link notifications.

Coverage label	What it means
Connected and enforceable	The adapter can observe and apply its documented restrictions.
Read-only	The source can provide evidence but cannot block activity.
Stale or offline	Current state is uncertain; show last observed time.
Unconnected	Saathi cannot claim visibility or control.
Owner-declared dependency	Useful context supplied by a person, with recorded freshness.

A clinic vocabulary pack does not automatically connect a clinic’s existing software. An integration must identify sessions, enforce decisions at protected operations and expose a way to check its outcomes. Without that, the claim is limited to visibility.

Permission is specific

Consent to temporary download throttling is not consent to disable accounts. Telegram linking is not authority to approve in chat. A device is enrolled to one organization and cannot accept commands for another device.

New organizations start with “No sources connected,” not a green “Protected” badge.

09 / DETECTION SCOPE

Threats and their actual responses

Patterns create reasons to investigate. They do not establish intent by themselves.

Observation	Proposed response	Boundary
Repeated failed logins	Review and preview supported lockout/source controls.	Locking a screen is not universal brute-force containment.
Unexpected storage	Review supported storage policy and affected device.	Already-mounted storage may remain accessible.
Fast records access	Temporary consented limit; offer session pause.	Low-and-slow extraction can evade simple thresholds.
Former staff request	Deny through connected access policy.	Unconnected accounts need separate offboarding.
Unresolved malware	Alert; offer supported scan/protection actions.	Scan completion is not proof of eradication.
Protection disabled	Preview restoration and check effective state.	Device management policy may prevent the change.
Decoy accessed	Review the context.	Not proof of malicious intent.
Related scam reports	Group the campaign; approve a staff warning.	Heuristics can overflag benign content.

Filtering without hiding evidence

Exact duplicates are removed; repeated failures retain counts; related signals are grouped. Known normal activity can reduce priority, but familiar devices must not suppress explicit threats. Every filtered item has a reason that can be inspected.

Initial thresholds in build.md are tunable prototype presets. Evaluate both suspicious and legitimate activity, and report false positives as well as missed threats. No accuracy percentage is assumed.

A large “events filtered” number is not evidence of good detection unless you also measure what was missed.

10 / FROM PRODUCT TO ENGINEERING

The complete system architecture

Next.js is the web and API layer. Neon is durable state. Separate processes do long-running work.



Component	Responsibility and connection
Next.js + TypeScript	Server-rendered pages and authenticated Route Handlers. Interactive UI calls validated APIs.
Neon + Drizzle + pg	Organization data, policy, incidents, actions, audit intents and durable jobs; transactional decisions.
Persistent worker	Claims jobs, correlates events, signs commands/evidence, notifies, reports and upgrades proofs.
Windows agent	Collects observations, polls over outbound HTTPS and executes only supported signed commands.
Records gateway	Checks every protected operation synchronously and reserves rate limits atomically.
Optional services	Telegram notifications, AI explanations, private object storage and OpenTimestamps.

Why a separate worker matters

A web request may end or time out. Security work must not disappear with it. State changes create audit intents and jobs in the same database transaction. A worker leases a job, performs bounded work, records results and retries safely after failures.

Where secrets live

Database credentials and AI tokens stay on server processes. Command and evidence private keys stay in worker secrets. The browser holds neither signing keys nor database credentials. The agent has only its device identity and pinned command-verification keys.

Implementation follows the installed Next.js guides and pins tested versions. Sensitive organization data is never placed in a shared cache.

11 / KEEP EVERY LAYER CONSISTENT

The data and API contract

One shared vocabulary prevents the interface and backend from making different promises.

Data family	Examples and purpose
Identity and scope	Organizations, memberships, staff, integrations, dependencies and policy versions.
Connected state	Devices, posture, accounts, application sessions, grants and rate counters.
Observations	Raw events, normalized signals, dispositions and incident findings.
Decisions	Stored previews, actions, commands, probes and recovery cases.
Reliability	Outbox, durable jobs, leases, attempts and delivery state.
Proof	Evidence intents, linked signed entries, checkpoints, anchor paths and proof files.

A browser action, traced through the layers

The preview button calls POST /api/v1/incidents/:id/previews. The API checks the current actor, loads tenant-scoped state, runs the shared policy engine and stores the preview. The browser displays the returned consequences. Approval sends only the preview ID and an idempotency key to POST /api/v1/actions.

The server rechecks authority and state, then commits the action and audit intent. The UI polls GET /api/v1/actions/:id for actual progress. Neither a frontend role flag nor an organization ID supplied in a request grants access.

Database protection

Every tenant-owned relationship is scoped to its organization. The data-access layer checks membership and returns minimal DTOs. Row-level policies add defense in depth, using transaction-local tenant context and restricted runtime roles. Cross-tenant tests must fail at every entry point.

Use one service layer from API handlers and server-side page loaders. Do not duplicate security rules in React components.

12 / POLICY, CONCURRENCY AND ENFORCEMENT

Making a preview trustworthy

The prediction must remain valid when the user actually approves it.

The policy model

A snapshot contains known subjects, sessions, grants, device posture, restrictions and dependencies. authorize(snapshot, request) returns allow, deny or throttle. A pure action simulation creates the proposed next state. Comparing the decisions produces stops, limits and unchanged capabilities.

The race to prevent

An owner opens an account-level preview. Before approval, another relevant session appears. If approval used the old snapshot, the display could omit a real consequence. Locking only the sessions that already existed does not prevent this new-session race.

The transaction protocol

- All relevant writers follow a common guard-row locking order, including session creation, grant changes and action approval.
- Approval rechecks current role, preview expiry, policy version and the relevant state hash inside the transaction.
- A relevant change yields STALE_PREVIEW and no action. Unrelated changes need not invalidate a narrow preview.
- A unique preview/action relation and idempotency key make duplicate clicks return one logical action.

Enforcement happens at the real boundary

Every protected records request passes through the same authorization function. Rate-budget reservation happens atomically before content is served. Detection jobs provide correlation, but asynchronous work is not allowed to be the only barrier against a burst of requests.

Evidence required

Property tests compare preview transitions with enforcement over randomized states. Database integration tests deliberately race approvals with session creation and simultaneous downloads. Windows previews have weaker guarantees: the actual device must be probed.

Prediction is exact only for the modeled connected policy. Unknown systems remain explicitly outside that guarantee.

13 / WINDOWS AGENT AND COMMANDS

The computer helper is bounded

The agent is powerful software. Its allowed behavior must be narrow, observable and testable.

A Windows agent reads selected event logs and posture. It batches authenticated observations, retains a bounded offline queue and reports its last contact time. Permissions and logging prerequisites must be checked on the actual supported Windows environment.

A command carries a contract

The signed envelope includes schema version, command ID, target organization/device, action ID, allowed type, validated parameters, expected local state, issue/expiry times, nonce and key ID. The agent rejects wrong-device, stale, replayed or invalid commands.

Control	What must be checked
Screen lock	Use an interactive-session helper and observe the intended session lock. A background service cannot assume it is the interactive desktop.
Storage policy	Check supported scope and effective behavior; disclose already-mounted and unsupported-device boundaries.
Account / firewall / AV	Read effective state and, where possible, test the intended behavior. A setting or rule merely existing is weaker evidence.
Undo	Restore saved prior state only if Saathi still owns the change; avoid overwriting later administrator edits.

Safe redelivery and crash recovery

Persist command receipt before execution. Repeated delivery returns the existing result. If the agent crashes after changing state, it reconciles the desired state on restart or reports unknown. It does not blindly repeat a potentially disruptive operation.

No arbitrary script or shell body comes from the server. Fixed adapters validate inputs, guard recovery access and protect the last enabled administrator. This reduces misuse paths; it does not make a privileged agent impossible to compromise.

Microsoft documents the asynchronous and interactive-desktop constraints of LockWorkStation; the implementation must follow those semantics. [S4]

14 / STATES, AUTOMATION AND RECOVERY

Action success is not incident success

Show what was observed, what is still uncertain and what the user can do next.

State family	Meaning
Action: requested / queued / executing	The response was accepted and is progressing; effectiveness is not yet established.
Action: confirmed / partial / failed / unknown	Postconditions observed, partly observed, failed, or not established.
Incident: open / investigating / contained	The broader situation, assessed against all known in-scope findings.
Incident: recovering / recovered	Recovery is underway or required checks have completed.
Evidence and anchor statuses	Separate integrity/timestamp progress, independent of containment.

Temporary automation has an explicit end

An initial policy may limit suspicious downloads to one per minute for 30 minutes. Its countdown, consequences and expiry behavior are shown before consent. At expiry, the temporary limit is lifted with an audit event and warning where needed. A separately approved pause or revocation remains in effect.

Recovery is a new verified workflow

- Confirm the legitimate staff member through a trusted channel.
- Address credentials and remaining incident findings as appropriate.
- Revoke the old paused application session and create a fresh one.
- Test new authorized access and rejection of the old token.
- Restore device policy only through a fresh scoped preview and probe.

If a probe times out, “unknown” is more honest than either “safe” or “nothing changed.”

New relevant evidence can reopen investigation. Dismissing an incident requires a reason and preserves its history.

15 / LANGUAGE, SCAMS AND ACCESSIBILITY

AI explains; policy authorizes

The core security workflow remains usable when the model is unavailable.

Everyday explanations

Core incident text is built from reviewed English, Kannada and Hindi templates with structured facts. Organization vocabulary changes the nouns. An optional AI explanation can add context, but it receives minimized facts and cannot call action tools or alter policy.

Ask Saathi

A staff member pastes a message or submits a bounded screenshot. Extracted text is shown for review and redaction before optional external processing. Deterministic checks inspect domains, known evidence and suspicious patterns; AI can explain those findings.

Assessment	How to interpret it
Known malicious	Reliable matching evidence supports the finding.
Suspicious	Warning signs exist, but identity or intent may be uncertain.
No known indicators	The checks did not recognize a problem; this does not guarantee safety.
Insufficient information	The input or available checks cannot support a useful decision.

A short URL or urgent wording alone is not proof of a scam. Do not fetch arbitrary submitted links in the prototype. Malicious instructions inside a message remain untrusted content and cannot reach an execution API.

Useful to non-experts

Use plain action labels, facts separated from guesses, clear touch targets, keyboard support and text beside colors. Read-aloud is offered only when the browser has a suitable voice. Missing audio must not make the explanation inaccessible.

If AI fails, deterministic findings and templates remain. If evidence is weak, show uncertainty rather than inventing reassurance.

16 / THE REST OF THE COMPLETE PRODUCT

Prevention, notifications and daily use

The product should be useful between incidents, not only during a demonstration.

A transparent health score

Device checks cover antivirus, firewall, updates, lockout, administrator exposure and remote access. Proposed weights total 100: 25, 25, 20, 10, 10 and 10 respectively. Each check can pass, fail or be unknown. Show unknown points and coverage separately.

The score describes observed posture, not the chance of avoiding a breach. A disconnected computer has stale information, not a freshly verified good score. Supported fixes use the same preview, approval and probe workflow; unsupported fixes show instructions and a recheck.

Notifications that respect context

Telegram delivers generic alerts with links to authenticated web pages. A link is not an approval token. Chat linking uses an expiring one-use code, and duplicate updates are deduplicated. Delivery failure is visible in the console and does not erase an incident.

A weekly digest

Summarize actual incidents, completed actions, unresolved recovery, already-remediated observations and coverage gaps. The report explains where counts came from. Staff campaign warnings are explicitly approved and sent only to selected authorized recipients.

Privacy in normal operation

Keep raw message attachments short-lived and private. Store minimized findings and tenant-keyed fingerprints where needed. Protected business content is not copied into telemetry by default. Retention settings are product decisions to validate with customers, not assumed legal compliance.

The homepage should answer: What needs me? What was handled? What can Saathi not see? What should I improve next?

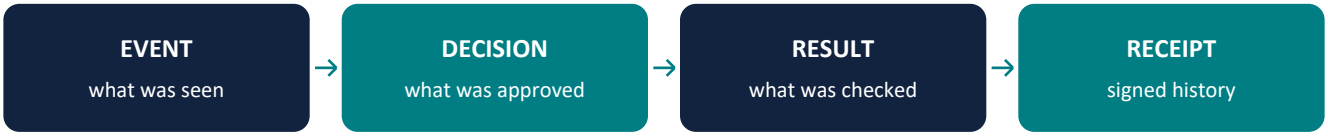
17 / THE RECEIPT

Signed evidence without the jargon

A useful record must show the decision, the action and the actual result.

Plain-language explanation

Imagine a numbered notebook where each entry refers to the fingerprint of the previous entry. Saathi also signs the entries with its evidence key. Changing an entry changes its fingerprint. A saved signed checkpoint tells the owner which final version they retained.



Technical construction

A mutation writes a durable evidence intent in the same transaction as its state change. The worker signs intents in organization sequence. Each canonical entry contains schema version, sequence, organization reference, type, event/record times, payload and previous hash. SHA-256 produces the digest; Ed25519 signs it.

Use distinct command and evidence keys. Database append restrictions resist ordinary edits, but privileged operators remain a threat. A checkpoint retained outside the server helps reveal truncation or rewriting that would otherwise leave a different internally consistent history.

What the download contains

A readable report, a canonical evidence manifest, signed entries, key IDs and checkpoint data. A PDF is the human view; its exact bytes need a separate signed export digest if the PDF itself is to be verified. The PDF must not attempt to contain its own final hash.

Privacy of evidence exports

The prototype can export a complete minimized chain for a synthetic organization to an authorized owner/auditor. A real incident-only export needs inclusion proofs or separate incident chains so unrelated incident details are not exposed.

Signed history is tamper-evident, not tamper-proof. Lost evidence cannot be reconstructed from a hash.

18 / INDEPENDENT TIMESTAMP

Where blockchain actually helps

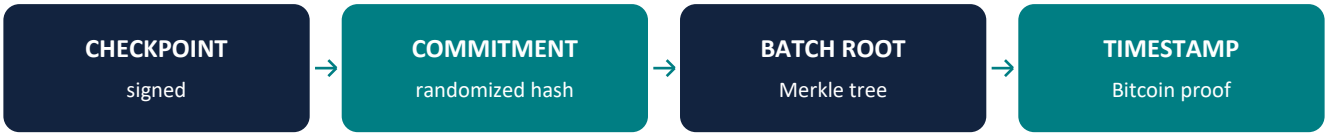
It adds an external reference for a retained version of the evidence.

The reason to include it

Suppose Saathi's server or signing key is compromised later. A previously confirmed public commitment gives the owner another reference against which to check retained evidence. It reduces reliance on the server's current story.

The selected integration

Use OpenTimestamps with its official client, rather than creating a token or custom smart contract. The protocol supports Bitcoin-backed timestamp proofs. The worker submits and later upgrades the proof; the customer does not need a wallet. [S5, S6]



How data stays out of the public record

Generate a random nonce and hash it with the canonical signed checkpoint using a versioned domain prefix. Batch these commitments into a defined Merkle tree. Stamp the exact batch-root file with OTS. Keep the nonce, checkpoint and inclusion path in the private proof bundle. Publish no names, raw logs or records.

Pending is a legitimate state

A calendar receipt means submitted, not confirmed on Bitcoin. The worker persists it, retries upgrades, verifies the eventual attestation and records the verification method. A validating Bitcoin node supports independent verification; a third-party verification service adds a trust dependency.

Recheck relevant chain changes rather than treating any downloaded proof as permanently valid. A block reference does not establish the exact time an incident occurred.

Timestamping runs after the security decision. Detection, enforcement and recovery never wait for blockchain confirmation.

19 / PROOF AND ITS LIMITS

What independent verification checks

Show each verification result separately so a green badge cannot hide a missing check.

Check	What it establishes
Hashes and sequence links	The supplied entries are internally consistent and ordered as declared.
Signatures and trusted keys	Entries verify under independently trusted signer keys, not a key blindly supplied by the bundle.
Retained checkpoint	The supplied history matches the particular final version retained by the owner.
Commitment and Merkle path	That checkpoint belongs to the timestamped batch-root file.
OTS plus valid chain context	The commitment is supported by the verified Bitcoin timestamp attestation.

Offline and online are different promises

A downloaded standalone verifier can check hashes, signatures, checkpoints and Merkle membership locally. The web verifier must already be available before going offline. Without independently validated Bitcoin chain context, it must say the blockchain timestamp was not checked offline.

A proof does not establish everything

- It does not prove that an original observation was truthful or that no event was omitted.
- It does not prove the response stopped an attack, or protect unconnected applications.
- It does not prove an exact incident time, provide legal compliance or restore deleted bundles.

A useful demonstration

Verify an earlier confirmed synthetic bundle, then edit a copy and show which check fails. For a new live incident, show its real pending timestamp state. A transaction explorer link may help orientation, but cannot replace proof verification.

The blockchain is a trust reference for evidence. The policy engine and probes establish the protective behavior.

20 / DELIVERY PLAN

Build the complete product in order

Each stage must produce a working, testable increment.

Order	Deliverable and dependency
1. Prove the risks	Windows event/lock spike and early synthetic timestamp request; settle the threat model.
2. Establish foundations	Next.js workspace, Neon migrations, identity, tenancy, roles and environment validation.
3. Make work durable	Audit intents, outbox, job leases, retries and observable failures.
4. Build enforcement	Reference records app, shared authorization, atomic rate budget and two independent sessions.
5. Observe real sources	Agent enrollment, authenticated event ingestion, heartbeat, filtering and correlation.
6. Close the response loop	Consequence preview, stale-state checks, approval, bounded commands and probes.
7. Complete the experience	Recovery, mobile UI, localization, Telegram, Ask Saathi and posture fixes.
8. Deliver proof	Signed evidence, standalone verifier, OTS background workflow and report downloads.
9. Validate and release	Failure tests, user checks, monitoring, restoration, rehearsal and honest scope labels.

Suggested human team ownership

One person owns UI/localization; one owns policy/database/API; one owns Windows integration; one owns evidence/anchoring/evaluation. Agree schemas first and assign an integration lead. This is a collaboration suggestion, not an assertion that every feature fits into 24 hours.

The companion build.md expands these into 19 numbered phases with implementation instructions, API endpoints, logical tables, gates, deployment controls and a complete acceptance matrix.

Build the core proof early: suspicious session denied, legitimate session working, failed actions reported honestly.

21 / EVALUATION AND DEMONSTRATION

How to prove it works

Demonstrate correctness and usability; report measurements instead of invented performance claims.

Test family	Evidence to collect
Policy correctness	Randomized preview/enforcement comparisons including allow, deny and throttle.
Concurrency	Racing approvals, session creation and download bursts; no hidden bypass.
Device execution	Real postconditions; expired, tampered and wrong-device commands rejected.
Recovery	Fresh access works and the old token stays denied.
Detection and scams	Normal and suspicious cases; false positives, misses and uncertainty recorded.
Evidence	Edited, reordered, truncated and wrong-key bundles fail; pending proof stays pending.
Human use	Non-experts choose a response and explain who loses or keeps access.

Measurements worth showing

Event-to-incident latency, approval-to-probe latency, legitimate-access preservation, false-alert/missed-threat counts, retry recovery and user decision accuracy. Record environment, sample size and failures. A small pilot is useful descriptive evidence, not proof of general effectiveness.

A three-minute story

Show a connected organization, cause suspicious records access, open the clear incident, compare consequences, pause one session and demonstrate that the other still works. Show a stale/failed case. Close with evidence verification and an earlier confirmed timestamp, clearly labelled.

Use a real Windows observation/action in the longer technical review. Keep the primary story focused. Show other organization vocabulary briefly, without implying new connector coverage.

No tool choice or extra feature guarantees a winning score. The strongest presentation makes the product's central claim visibly true.

22 / FROM PROTOTYPE TO REAL USE

Boundaries that make the idea credible

A complete explanation includes what cannot be promised.

Limits of the prototype

- Only enrolled devices and connected applications are visible. Some connectors are observation-only.
- Already copied information cannot be recalled. Slow extraction can stay below simple thresholds.
- Device policy behavior varies; unsupported actions remain unavailable.
- Declared dependencies can be incomplete. Show unknown effects.
- A compromised endpoint can lie about its observations; signatures identify a source, not its honesty.
- AI and heuristics can be wrong. Evidence integrity and blockchain timestamps do not make those judgments true.

Before real customer deployment

Complete a security review, tenant-isolation assessment, signed agent distribution/update design, hardened key management, backup restoration, connector validation, customer support, language/accessibility review and retention agreements. Test on the actual supported environment with authorized users.

Roadmap that follows demonstrated capability

Add read-only cloud signals before promising cloud enforcement. Add narrowly scoped connectors only when their APIs can support the claimed controls. Consider WhatsApp, more languages, chain-level views and authorized device discovery after the central workflow is dependable.

Validate demand across clinics, shops, schools, NGOs and offices. A per-office subscription or supported community tier is a business hypothesis to test, not an established business model.

Saathi’s promise is understandable, scoped and verified security decisions. It should never claim to be foolproof.

23 / SOURCES

References and implementation handoff

Primary documentation supports platform behavior; the architecture itself is the proposed Saathi design.

Source concept: VAJRA_Saathi_Explained.pdf, supplied by the user, 17 pages. This edition generalizes the product beyond schools, incorporates the agreed blockchain extension, and refines verification, uncertainty and containment semantics.

Primary references

[S1] Kodikon 6 official event and Track 3 description

[S2] Next.js Route Handlers and data security. Installed package guides were consulted for version-sensitive conventions.

[S3] Neon connection pooling and Better Auth session management.

[S4] Microsoft LockWorkStation and device-installation policy guidance.

[S5] OpenTimestamps protocol and service overview.

[S6] Official OpenTimestamps client for stamping, proof upgrades and verification.

Engineering handoff

build.md: complete implementation sequence, stack, architecture diagram, table model, API inventory, policy locking protocol, agent contract, proof construction, tests and release gates.

flow.md: complete screen map, role navigation, onboarding, incident decisions, device actions, Ask Saathi, health, evidence, recovery and failure paths.

How to interpret this document

All thresholds, example flows and acceptance criteria are proposed product decisions. No benchmark result, production readiness, legal-compliance status or competition outcome is asserted. Recheck installed package guides and service behavior when implementing.

Prepared 31 October 2026. All scenarios use invented organizations, people and records.